

Extra_CG_LAB 1 – Assignment

(2023BCS063, 2023BCS064, 2023BCS044, 2023BCS081)

Objective: To design and implement a virtual 3D environment that demonstrates the practical application of computer graphics concepts, using any programming language, graphics API, or development framework.

Details: In this assignment, you are required to create a virtual environment of your choice (e.g., a room, a cityscape, a natural landscape, a virtual museum, etc.). The environment should be visually coherent, incorporating appropriate lighting, textures, shading, and camera controls.

Solution:-

Of course. Here is a short procedure for creating the "Terrain Trails 3D" project, based on the provided source code and project description.

Repo link: <https://github.com/chansom63/3D-Terrain-Visualizer.git>

1. Environment Setup

- Create a basic HTML file to serve as the application's entry point.
- Include the core **Three.js** library for 3D rendering.
- Include all the project's JavaScript modules: core.js, generators.js, filters.js, analysis.js, and others.

2. Core Scene Initialization

- Set up a standard Three.js environment: a Scene, a PerspectiveCamera, a WebGLRenderer, and basic lighting (e.g., AmbientLight and DirectionalLight).
- Implement camera controls (like OrbitControls) to allow users to navigate the 3D space.

3. Procedural Terrain Generation

- Define a set of configuration options that specify the terrain's properties, such as xSize, ySize, xSegments, maxHeight, and minHeight.
- Use the main THREE.Terrain() function from core.js to create the terrain object. For the heightmap option, pass one of the procedural generation algorithms defined in generators.js, such as THREE.Terrain.DiamondSquare or THREE.Terrain.Perlin.
- Add the resulting terrain mesh to the Three.js scene.

4. Interactive UI Panel

- Integrate a UI library (like dat.GUI) to create an interactive control panel.
- Link the UI controls to the terrain configuration options.
- Write a function to regenerate the terrain and update the scene whenever a user changes a parameter, enabling real-time customization.

5. Implementing Advanced Features

- **Terrain Analytics:** After generating a terrain mesh, call the THREE.Terrain.Analyze() function from analysis.js. This function takes the mesh and options as input and returns an object containing detailed statistics like elevation, slope, ruggedness, and jaggedness. Display this data in a dedicated analytics panel in the UI.
- **Texturing:** Use the THREE.Terrain.generateBlendedMaterial() function from materials.js to create a material that blends multiple textures based on the terrain's height and slope, giving it a more realistic appearance.
- **Filtering & Effects:** Apply post-generation filters from filters.js, such as THREE.Terrain.Smooth() for a softer look or THREE.Terrain.Step() to create a blocky, stylized terrain.

6. Final Rendering Loop

- Create an animation loop using requestAnimationFrame() to continuously render the scene, ensuring smooth camera movement and interaction.



